SOCIAL EAGLE
learn. build. operate

# CAIE Architect Program

Assignment 2 - Playwright Assignment | learn. build. operate

## 1. Scenario

You want to automate daily customer and business communication. Your task is to build a **Playwright-powered WhatsApp Web Automation Bot** that can send personalized messages to a list of contacts and also extract data back from chats. The bot drives a real browser through WhatsApp Web, just like a person would. This is a **medium-level** automation assignment.

*Title: WhatsApp Message Sender + Smart Data Extractor*
*Difficulty: Medium · Tools: Playwright (Python), WhatsApp Web, Excel*

## 2. What your bot must do

The bot should perform the following sequence:

- Log in to WhatsApp Web (handle the QR code manually on the first run).
- Read contacts from an Excel file named `contacts.xlsx` with columns: **Name**, **Phone** (with country code, e.g. +91xxxxxxxxxx), and **Message** (an optional template).
- For each contact: search the contact or number, send a personalized message (replace `{name}` with the actual name), wait for the message to be sent, then take a screenshot of the sent message.
- Smart data extraction (bonus core part): after sending, open a contact and extract the last 3 messages from them.
- Save everything in a report as both JSON and Excel.

> **Think it through**
> WhatsApp Web loads chats dynamically, so timing is everything. Plan where your bot needs to wait for elements to appear, how it confirms a message was actually sent, and how it behaves when a contact is not found. Sending too fast looks robotic and risks bans, so build in human-like pauses.

## 3. Must implement

- Proper waits using `wait_for_selector` and `wait_for_timeout`.
- Search a contact by number or name.
- Type and send a personalized message, handling the “Type a message” box reliably.
- Add random delays (2-5 seconds) between actions to avoid bans.
- Error handling (for example, contact not found) so the run does not crash.
- Save the sent status for each contact in the report.
- Keep all your code in a single file named `playwright_assign.py`.

## 4. Reports to produce

At the end of a run your bot must save two dated report files:

- `whatsapp_report_YYYY-MM-DD.json` (full details).
- `whatsapp_report_YYYY-MM-DD.xlsx` (a summary).

## 5. Step-by-step setup

Follow these steps in order. Commands are shown for both macOS/Linux and Windows where they differ.

### Step 1 - Create a project folder

```
mkdir playwright-whatsapp-bot
cd playwright-whatsapp-bot
```

### Step 2 - Create a virtual environment (venv)

A virtual environment keeps this project isolated from the rest of your system.

```
python -m venv venv
# (on some systems use 'python3' instead of 'python')
```

### Step 3 - Activate the virtual environment

```
# macOS / Linux:
source venv/bin/activate

# Windows (PowerShell):
venv\Scripts\Activate.ps1

# Windows (Command Prompt):
venv\Scripts\activate.bat
```

When activated, you will see `(venv)` at the start of your terminal prompt.

### Step 4 - Install Playwright and its browsers

Install the Playwright library, then download the browser binaries it needs. Add any helper libraries (such as one for reading and writing Excel) that you decide to use.

```
pip install playwright
playwright install

# add any other libraries your bot needs, e.g.:
# pip install openpyxl
```

### Step 5 - Run your program

```
python playwright_assign.py
```

On the first run, scan the QR code with your phone to log in to WhatsApp Web.

**Step 6 - Deactivate when finished**

```
deactivate
```

## 6. What to submit

- Your `playwright_assign.py` file.
- A short screen recording (30-60 seconds) showing the bot working.
- The sample output reports (`.json` and `.xlsx`) and the saved screenshots.
- Upload the code to GitHub and send us the repository link.

---

Reminder: build the automation logic yourself - the steps above are your setup roadmap, not the answer. Use this responsibly and only message people who have agreed to hear from you. Good luck!